MA388 Sabermetrics: Lesson 21

Simulating Baseball - Part I

Lesson Objectives

1. Construct a transition probability matrix from Retrosheet play-by-play data.
2. Model a baseball half-inning as a Markov chain and simulate 10,000 half-innings.
3. Compute the distribution of runs scored per half-inning and the average runs per half-inning.
4. Discuss the memoryless property of Markov chains and whether it allows for momentum.

Markov Chains

Markov chains model the probabilities of moving between different states. I want you to draw a state diagram (think a network diagram with states as circles and lines connecting the states with probabilities) for a baseball half-inning based on the number of outs where probabilities reflect the state transitions after one plate appearance. Assume that every plate appearance has a .7 probability of resulting in an out and .3 probability of not and there is never more than one out for a plate appearance.

What is the probability of having 0 outs after two plate appearances? What about having two outs?

What is the memoryless property of Markov chains? Does this allow for "momentum"?

So this was all very basic, but hopefully a useful intro. What if we wanted to model baseball this way to determine how many at-bats occur in a given baseball half-inning (or entire game)? How might you approach this using Markov Chains and the data we have? How would you change the states?

```
library(tidyverse)
library(knitr)
library(baseballr)

# Load the data.
retro2024 <-
retrosheet_data(years_to_acquire = '2024') |>
  pluck('2024') |>
  pluck('events')
```

Let's add states, etc., similarly to how we've done it in previous lessons. One important difference is that we don't care if/where baserunners were left on base when the third out was made. The third out is, however, an important state for us to capture.

```r
# Add states, etc.
retro2024 <- retro2024 |>
  mutate(runs_before = away_score_ct + home_score_ct,
         half_inning = paste(game_id, inn_ct, bat_home_id),
         runs_scored =
           (bat_dest_id > 3) + (run1_dest_id > 3) +
           (run2_dest_id > 3) + (run3_dest_id > 3))

half_innings <- retro2024 |>
  summarize(outs_inning = sum(event_outs_ct),
            runs_inning = sum(runs_scored),
            runs_start = first(runs_before),
            max_runs = runs_inning + runs_start, .by ='half_inning')

retro2024_complete <- retro2024 |>
  inner_join(half_innings, by = "half_inning") |>
  mutate(runs_roi = max_runs - runs_before) |>
  mutate(bases =
           paste(ifelse(base1_run_id > '',1,0),
                 ifelse(base2_run_id > '',1,0),
                 ifelse(base3_run_id > '',1,0), sep = ""),
         state = paste(bases, outs_ct)) |>
  mutate(is_runner1 =
           as.numeric(run1_dest_id==1 | bat_dest_id == 1),
         is_runner2 =
           as.numeric(run1_dest_id == 2 | run2_dest_id == 2 |
                        bat_dest_id == 2),
         is_runner3 =
           as.numeric(run1_dest_id == 3 | run2_dest_id == 3 |
                        run3_dest_id == 3 | bat_dest_id == 3),
         new_outs = outs_ct + event_outs_ct,
         new_bases = paste(is_runner1, is_runner2, is_runner3, sep = ""),
         new_state = paste(new_bases, new_outs)) |>
  filter((state != new_state) | (runs_scored > 0)) |>
  filter(outs_inning == 3, bat_event_fl == TRUE) |>
  mutate(new_state = str_replace(new_state, "[0-1]{3} 3", "3"))
```

Let's look at our resulting data frame:

```
retro2024_complete |>
  select(game_id, bat_id, state, new_state) |>
  head(10) |>
  kable()
```

game_id	bat_id	state	new_state
ANA202404050	duraj001	000 0	000 1
ANA202404050	dever001	000 1	100 1
ANA202404050	stort001	100 1	100 2
ANA202404050	yoshm002	100 2	3
ANA202404050	renda001	000 0	000 1
ANA202404050	schan001	000 1	000 2
ANA202404050	troum001	000 2	3
ANA202404050	oneit001	000 0	000 0
ANA202404050	casat001	000 0	000 0
ANA202404050	valde002	000 0	000 1

We're now ready to compute a matrix of transition probabilities. How many starting states are there? What about ending states? How can we compute these state-to-state probabilities?

```
T_matrix <- retro2024_complete  |>
  select(state, new_state) |>
  table()
T_matrix
```

```
       new_state
state   000 0 000 1 000 2 001 0 001 1 001 2 010 0 010 1 010 2 011 0 011 1 011 2
  000 0  1381 30649     0   210     0     0  1983     0     0     0     0     0
  000 1     0   961 22729     0   145     0     0  1390     0     0     0     0
  000 2     0     0   725     0     0    87     0     0  1154     0     0     0
  001 0    13    80     4     2   175     0    20     1     0     0     0     0
  001 1     0    50   340     0     5   615     0    84    10     0     0     0
  001 2     0     0    67     0     0    13     0     0    97     0     0     0
  010 0    76     8    18    13   788     0   146  1460     0    14     0     0
  010 1     0   139    24     0    20   905     0   226  2398     0    14     0
  010 2     0     0   203     0     0    37     0     0   344     0     0     2
```

011	0	22	1	0	2	91	2	44	53	2	3	251	0
011	1	0	26	3	0	8	235	0	97	125	0	6	469
011	2	0	0	55	0	0	13	0	0	73	0	0	0
100	0	293	5	1111	43	31	0	120	672	0	358	0	0
100	1	0	402	7	0	68	27	0	137	749	0	369	0
100	2	0	0	351	0	0	70	0	0	237	0	0	297
101	0	30	2	50	5	3	10	10	37	0	35	22	0
101	1	0	48	1	0	11	8	0	19	92	0	80	46
101	2	0	0	72	0	0	6	0	0	55	0	0	58
110	0	74	1	0	12	3	206	25	2	20	94	227	0
110	1	0	140	2	0	18	3	0	64	11	0	158	235
110	2	0	0	146	0	0	34	0	0	117	0	0	140
111	0	26	1	0	4	0	48	6	2	3	21	45	9
111	1	0	36	1	0	12	1	0	20	5	0	54	86
111	2	0	0	66	0	0	14	0	0	51	0	0	33

	new_state											
state	100 0	100 1	100 2	101 0	101 1	101 2	110 0	110 1	110 2	111 0	111 1	111 2
000 0	10248	0	0	0	0	0	0	0	0	0	0	0
000 1	0	7558	0	0	0	0	0	0	0	0	0	0
000 2	0	0	6070	0	0	0	0	0	0	0	0	0
001 0	63	2	0	52	0	0	0	0	0	0	0	0
001 1	0	296	55	0	214	0	0	0	0	0	0	0
001 2	0	0	337	0	0	347	0	0	0	0	0	0
010 0	167	32	0	298	0	0	410	0	0	0	0	0
010 1	0	301	47	0	325	0	0	633	0	0	0	0
010 2	0	0	598	0	0	178	0	0	963	0	0	0
011 0	35	3	0	47	2	0	5	4	0	68	0	0
011 1	0	104	5	0	110	43	0	10	16	0	244	0
011 2	0	0	176	0	0	56	0	0	0	0	0	303
100 0	1	4640	0	385	0	0	1964	0	0	0	0	0
100 1	0	1	5855	0	506	0	0	2306	0	0	0	0
100 2	0	0	10	0	0	631	0	0	2102	0	0	0
101 0	0	154	2	34	237	0	127	18	0	64	0	0
101 1	0	0	322	0	68	472	0	244	34	0	151	0
101 2	0	0	1	0	0	137	0	0	203	0	0	189
110 0	0	1	17	65	257	0	101	986	0	447	0	0
110 1	0	0	14	0	146	448	0	171	1726	0	624	0
110 2	0	0	2	0	0	255	0	0	214	0	0	668
111 0	0	3	2	15	73	4	23	49	1	108	235	0
111 1	0	0	4	0	33	144	0	51	128	0	217	493
111 2	0	0	0	0	0	99	0	0	75	0	0	236

	new_state
state	3
000 0	0
000 1	0
000 2	18355

```
  001 0     0
  001 1    11
  001 2  1817
  010 0     0
  010 1    20
  010 2  4419
  011 0     0
  011 1    30
  011 2  1315
  100 0     0
  100 1  1283
  100 2  8023
  101 0     1
  101 1   192
  101 2  1544
  110 0     1
  110 1   483
  110 2  3656
  111 0     0
  111 1   161
  111 2  1280
```

```
P_matrix <- prop.table(T_matrix, 1)
dim(P_matrix)
```

```
[1] 24 25
```

Once you get to three outs, there's nowhere left to go – you will stay in three outs with probability 1.

```
P_matrix <- P_matrix |>
  rbind("3" = c(rep(0, 24), 1))
# dim(P_matrix)
P_matrix |> round(3)
```

```
      000 0 000 1 000 2 001 0 001 1 001 2 010 0 010 1 010 2 011 0 011 1 011 2
000 0 0.031 0.689 0.000 0.005 0.000 0.000 0.045 0.000 0.000 0.000 0.000 0.000
000 1 0.000 0.029 0.693 0.000 0.004 0.000 0.000 0.042 0.000 0.000 0.000 0.000
000 2 0.000 0.000 0.027 0.000 0.000 0.003 0.000 0.000 0.044 0.000 0.000 0.000
001 0 0.032 0.194 0.010 0.005 0.425 0.000 0.049 0.002 0.000 0.000 0.000 0.000
001 1 0.000 0.030 0.202 0.000 0.003 0.366 0.000 0.050 0.006 0.000 0.000 0.000
001 2 0.000 0.000 0.025 0.000 0.000 0.005 0.000 0.000 0.036 0.000 0.000 0.000
010 0 0.022 0.002 0.005 0.004 0.230 0.000 0.043 0.426 0.000 0.004 0.000 0.000
010 1 0.000 0.028 0.005 0.000 0.004 0.179 0.000 0.045 0.475 0.000 0.003 0.000
```

```
010 2 0.000 0.000 0.030 0.000 0.000 0.005 0.000 0.000 0.051 0.000 0.000 0.000
011 0 0.035 0.002 0.000 0.003 0.143 0.003 0.069 0.083 0.003 0.005 0.395 0.000
011 1 0.000 0.017 0.002 0.000 0.005 0.153 0.000 0.063 0.082 0.000 0.004 0.306
011 2 0.000 0.000 0.028 0.000 0.000 0.007 0.000 0.000 0.037 0.000 0.000 0.000
100 0 0.030 0.001 0.115 0.004 0.003 0.000 0.012 0.070 0.000 0.037 0.000 0.000
100 1 0.000 0.034 0.001 0.000 0.006 0.002 0.000 0.012 0.064 0.000 0.032 0.000
100 2 0.000 0.000 0.030 0.000 0.000 0.006 0.000 0.000 0.020 0.000 0.000 0.025
101 0 0.036 0.002 0.059 0.006 0.004 0.012 0.012 0.044 0.000 0.042 0.026 0.000
101 1 0.000 0.027 0.001 0.000 0.006 0.004 0.000 0.011 0.051 0.000 0.045 0.026
101 2 0.000 0.000 0.032 0.000 0.000 0.003 0.000 0.000 0.024 0.000 0.000 0.026
110 0 0.029 0.000 0.000 0.005 0.001 0.081 0.010 0.001 0.008 0.037 0.089 0.000
110 1 0.000 0.033 0.000 0.000 0.004 0.001 0.000 0.015 0.003 0.000 0.037 0.055
110 2 0.000 0.000 0.028 0.000 0.000 0.006 0.000 0.000 0.022 0.000 0.000 0.027
111 0 0.038 0.001 0.000 0.006 0.000 0.071 0.009 0.003 0.004 0.031 0.066 0.013
111 1 0.000 0.025 0.001 0.000 0.008 0.001 0.000 0.014 0.003 0.000 0.037 0.059
111 2 0.000 0.000 0.036 0.000 0.000 0.008 0.000 0.000 0.028 0.000 0.000 0.018
3     0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000
      100 0 100 1 100 2 101 0 101 1 101 2 110 0 110 1 110 2 111 0 111 1 111 2
000 0 0.230 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000
000 1 0.000 0.231 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000
000 2 0.000 0.000 0.230 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000
001 0 0.153 0.005 0.000 0.126 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000
001 1 0.000 0.176 0.033 0.000 0.127 0.000 0.000 0.000 0.000 0.000 0.000 0.000
001 2 0.000 0.000 0.126 0.000 0.000 0.130 0.000 0.000 0.000 0.000 0.000 0.000
010 0 0.049 0.009 0.000 0.087 0.000 0.000 0.120 0.000 0.000 0.000 0.000 0.000
010 1 0.000 0.060 0.009 0.000 0.064 0.000 0.000 0.125 0.000 0.000 0.000 0.000
010 2 0.000 0.000 0.089 0.000 0.000 0.026 0.000 0.000 0.143 0.000 0.000 0.000
011 0 0.055 0.005 0.000 0.074 0.003 0.000 0.008 0.006 0.000 0.107 0.000 0.000
011 1 0.000 0.068 0.003 0.000 0.072 0.028 0.000 0.007 0.010 0.000 0.159 0.000
011 2 0.000 0.000 0.088 0.000 0.000 0.028 0.000 0.000 0.000 0.000 0.000 0.152
100 0 0.000 0.482 0.000 0.040 0.000 0.000 0.204 0.000 0.000 0.000 0.000 0.000
100 1 0.000 0.000 0.500 0.000 0.043 0.000 0.000 0.197 0.000 0.000 0.000 0.000
100 2 0.000 0.000 0.001 0.000 0.000 0.054 0.000 0.000 0.179 0.000 0.000 0.000
101 0 0.000 0.183 0.002 0.040 0.282 0.000 0.151 0.021 0.000 0.076 0.000 0.000
101 1 0.000 0.000 0.180 0.000 0.038 0.264 0.000 0.136 0.019 0.000 0.084 0.000
101 2 0.000 0.000 0.000 0.000 0.000 0.060 0.000 0.000 0.090 0.000 0.000 0.083
110 0 0.000 0.000 0.007 0.026 0.101 0.000 0.040 0.388 0.000 0.176 0.000 0.000
110 1 0.000 0.000 0.003 0.000 0.034 0.106 0.000 0.040 0.407 0.000 0.147 0.000
110 2 0.000 0.000 0.000 0.000 0.000 0.049 0.000 0.000 0.041 0.000 0.000 0.128
111 0 0.000 0.004 0.003 0.022 0.108 0.006 0.034 0.072 0.001 0.159 0.347 0.000
111 1 0.000 0.000 0.003 0.000 0.023 0.100 0.000 0.035 0.089 0.000 0.150 0.341
111 2 0.000 0.000 0.000 0.000 0.000 0.053 0.000 0.000 0.040 0.000 0.000 0.127
3     0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000 0.000
         3
000 0 0.000
000 1 0.000
```

```
000 2 0.696
001 0 0.000
001 1 0.007
001 2 0.678
010 0 0.000
010 1 0.004
010 2 0.655
011 0 0.000
011 1 0.020
011 2 0.660
100 0 0.000
100 1 0.110
100 2 0.684
101 0 0.001
101 1 0.107
101 2 0.682
110 0 0.000
110 1 0.114
110 2 0.699
111 0 0.000
111 1 0.111
111 2 0.690
3     1.000
```

How can we check the validity of this transition matrix? Pick a state – what should the sum of all transition probabilities from that state to any other be?

```
P_matrix |>
  apply(MARGIN = 1, FUN = sum)
```

```
000 0 000 1 000 2 001 0 001 1 001 2 010 0 010 1 010 2 011 0 011 1 011 2 100 0
    1     1     1     1     1     1     1     1     1     1     1     1     1
100 1 100 2 101 0 101 1 101 2 110 0 110 1 110 2 111 0 111 1 111 2     3
    1     1     1     1     1     1     1     1     1     1     1     1
```

Let's examine the nobody-on, nobody-out state. What are the most likely resulting states?

```
P_matrix |>
  as_tibble(rownames = "state")  |>
  filter(state == "000 0") |>
  pivot_longer(
    cols = -state,
    names_to = "new_state",
    values_to = "Prob"
  ) |>
  filter(Prob > 0) |>
  kable()
```

state	new_state	Prob
000 0	000 0	0.0310539
000 0	000 1	0.6891907
000 0	001 0	0.0047222
000 0	010 0	0.0445909
000 0	100 0	0.2304423

What about with a runner on 2nd base and two outs?

```
P_matrix |>
  as_tibble(rownames = "state") |>
```

```
  filter(state == "010 2") |>
  pivot_longer(
    cols = -state,
    names_to = "new_state",
    values_to = "Prob"
  ) |>
  filter(Prob > 0) |>
  kable()
```

state	new_state	Prob
010 2	000 2	0.0301008
010 2	001 2	0.0054864
010 2	010 2	0.0510083
010 2	011 2	0.0002966
010 2	100 2	0.0886714
010 2	101 2	0.0263938
010 2	110 2	0.1427936
010 2	3	0.6552491

So far, so good… Now we need to actually simulate the Markov Chain. This involves being in a state and transitioning into possible other states in a probabilistic manner…a large number of times.

The key equation for keeping score is:

$$runs = \left(N_{runners}^{(before)} + Outs^{(before)} + 1\right) - \left(N_{runners}^{(after)} + Outs^{(after)}\right)$$

```
# This function essentially "adds" the state: "011 1" becomes 3.
num_havent_scored <- function(s) {
  s |>
    str_split("") |>
    pluck(1) |>
    as.numeric() |>
    sum(na.rm = TRUE)
}

# "Add" the state (runners + outs) for every state.
runners_out <- T_matrix |>
  row.names() |>
  set_names() |>
  map_int(num_havent_scored)

# This operation computes the runs equation above for every state-to-state
```

```
# transition, including the impossible transitions that we can safely ignore.
R_runs <- outer(
  runners_out + 1,
  runners_out,
  FUN = "-"
) |>
  cbind("3" = rep(0, 24))
```

Before we go any further, what are the key tasks for simulating a half-inning of baseball?

Here we go - let's simulate!

```
# This function returns the number of runs scored in a simulated half-inning.
simulate_half_inning <- function(P, R, start = 1) {
  s <- start
  path <- NULL
  runs <- 0
  while (s < 25) {
    s_new <- sample(1:25, size = 1, prob = P[s, ])
    path <- c(path, s_new)
    runs <- runs + R[s, s_new]
    s <- s_new
  }
  runs
}
```

Let's simulate 10,000 half-innings.

```
set.seed(388)
simulated_runs <- 1:10000 |>
  map_int(~simulate_half_inning(P_matrix, R_runs))
P_matrix[1,]
```

```
      000 0       000 1       000 2       001 0       001 1       001 2
0.031053945 0.689190709 0.000000000 0.004722178 0.000000000 0.000000000
      010 0       010 1       010 2       011 0       011 1       011 2
0.044590857 0.000000000 0.000000000 0.000000000 0.000000000 0.000000000
      100 0       100 1       100 2       101 0       101 1       101 2
```

```
0.230442311 0.000000000 0.000000000 0.000000000 0.000000000 0.000000000
      110 0       110 1       110 2       111 0       111 1       111 2
0.000000000 0.000000000 0.000000000 0.000000000 0.000000000 0.000000000
          3
0.000000000
```

What was the observed distribution of half-inning run totals?

```
table(simulated_runs)
```

```
simulated_runs
   0    1    2    3    4    5    6    7    8
7435 1411  653  279  120   59   21   17    5
```

How often do teams score at least five runs in an inning?

```
sum(simulated_runs >= 5) / 10000
```

```
[1] 0.0102
```

What is the average number of runs scored per half-inning?

```
mean(simulated_runs)
```

```
[1] 0.4614
```

What have we left out? Name some non-batting plays that have the potential to change the state and potentially score runs.